

OCA I2C Controller — Low-Level Design

SMC I2C Controller — Low-Level SystemC Implementation

Contents

1. Purpose and scope
2. Source layout
3. Module structure
4. TLM-2.0 interface implementation
 - 4.1 `b_transport`
 - 4.2 `transport_dbg`
5. Register decode
6. Internal data structures
7. Process and event topology
8. Behavioural algorithms
 - 8.1 `push_fmt(fdata)` (FDATA write)
 - 8.2 `drain_fmt()` (xfer engine)
 - 8.3 Target back doors
 - 8.4 Status / level computation
9. Reset implementation
10. Loosely-timed timing model and quantum keeper
11. Interrupt aggregation
12. Debug and verification hooks
13. Integration with `smc_top`
14. Modeling decisions and trade-offs
15. Known limitations and future work

SMC I2C Controller — Low-Level SystemC Implementation

Document: 02_I2C_CONTROLLER_LowLevel_Design.md **Module:** smc::i2c_controller
(peripherals/i2c_controller/) **SystemC:** 3.0.x Accellera (compatible with $\geq 2.3.4$), TLM-2.0, CCI 1.0 **RTL reference:** hw/comp/i2c/ (register map hw/comp/i2c/data/registers/rd1/i2c.rdl) **Status:** Matches
include/i2c_controller.h and src/i2c_controller.cpp **Companion docs:** -
01_I2C_CONTROLLER_Specification.md — externally-observable behaviour -
03_I2C_CONTROLLER_Test_Plan.md — verification strategy and test list

Contents

1. Purpose and scope
 2. Source layout
 3. Module structure
 4. TLM-2.0 interface implementation
 5. Register decode
 6. Internal data structures
 7. Process and event topology
 8. Behavioural algorithms
 9. Reset implementation
 10. Loosely-timed timing model and quantum keeper
 11. Interrupt aggregation
 12. Debug and verification hooks
 13. Integration with smc_top
 14. Modeling decisions and trade-offs
 15. Known limitations and future work
-

1. Purpose and scope

01_I2C_CONTROLLER_Specification.md defines **what** the I2C core looks like from the outside — sockets, signals, register layout, protocol semantics. This document defines **how** the SystemC/TLM-2.0 loosely-timed functional model is implemented internally:

- the data structures it owns,
- the SystemC process topology that drives them,
- the algorithms for the FMT-drain engine, target back doors, and interrupt aggregation,
- the loosely-timed timing strategy and the role of the quantum keeper,
- the verification hooks exposed to the test bench, and
- the integration recipe required to drop the model into smc_top.

It is the implementation contract that any future maintainer should be able to read in isolation.

2. Source layout

```
peripherals/i2c_controller/
├─ CMakeLists.txt
├─ README.md
├─ run_tests.sh           Build + run convenience script
├─ deps.env.example       Template for local dependency paths
├─ doc/
│   └─ 01_I2C_CONTROLLER_Specification.md
│   └─ 02_I2C_CONTROLLER_LowLevel_Design.md   (this file)
│   └─ 03_I2C_CONTROLLER_Test_Plan.md
├─ include/
│   └─ smc_tlm_extensions.h   Shared GP extension (smc_axi_extension)
│   └─ i2c_controller.h       SC_MODULE(i2c_controller) declaration
├─ src/
│   └─ i2c_controller.cpp      Implementation
└─ test/
    ├── CMakeLists.txt
    ├── i2c_controller_tb.cpp   Primary self-checking test bench
    └─ i2c_controller_neg_tb.cpp Negative-path / edge-case test bench
```

The model reuses the shared SMC helpers:

- `common/include/reg_access.h` — `regmodel::Register<N>` typed register wrappers (read/write mask + reset contract, `apply_w1c`, `bit`).
- `common/include/reg_map.h` — `regmodel::RegisterMap<N>` offset → register dispatch table.
- `common/include/sim_log.h` — leveled `SIM_LOG_*` logging over `sc_report`.

3. Module structure

`class smc::i2c_controller : public sc_core::sc_module` (declared in `include/i2c_controller.h`).

Declaration order matters: the six `cci::cci_param<>` members are declared **first** so their values are resolved before the sized state (FIFO depths) is used.

```
class i2c_controller : public sc_core::sc_module {
protected:
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> fmt_fifo_depth_p_;
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> rx_fifo_depth_p_;
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> tx_fifo_depth_p_;
    cci::cci_param<unsigned, cci::CCI_IMMUTABLE_PARAM> acq_fifo_depth_p_;
    cci::cci_param<double> access_delay_ns_p_;
    cci::cci_param<double> xfer_delay_ns_p_;
public:
    tlm_utils::simple_target_socket<i2c_controller> reg_socket;
    sc_core::sc_in<bool> rst_n_i;
    sc_core::sc_out<bool> irq_o;
    ...
};
```

The constructor:

1. Constructs each CCI param with the corresponding `cfg` field as its default and attaches `rtl_param / unit` metadata for introspection.
2. Snapshots the resolved FIFO depths into `cfg_` and validates them (`SC_REPORT_FATAL` if any is 0).
3. Builds the `regmap_` dispatch table for the plain storage registers.
4. Registers `b_transport / transport_dbg` on `reg_socket`.
5. Registers the three `SC_METHODS` (`reset_proc`, `recompute_method`, `xfer_method`), each with `dont_initialize()`.
6. Emits an `SC_REPORT_INFO` summary of the resolved configuration.

4. TLM-2.0 interface implementation

4.1 `b_transport`

```

validate command → TLM_COMMAND_ERROR_RESPONSE if not READ/WRITE
validate length  → TLM_BURST_ERROR_RESPONSE   if len != 4
validate address → TLM_ADDRESS_ERROR_RESPONSE if unaligned or >= WINDOW_SIZE
dispatch        → reg_read()/reg_write()
                  → TLM_OK_RESPONSE           on decode hit
                  → TLM_ADDRESS_ERROR_RESPONSE on in-window decode miss
delay += access_delay_ns
set_dmi_allowed(false)

```

The optional `smc::smc_axi_extension` is fetched but not enforced (upstream `axi_filter` owns access control).

4.2 `transport_dbg`

Rejects malformed accesses (length $\neq$ 4, unaligned, out-of-window) by returning 0. Reads go through `dbg_reg()` (side-effect-free peek); writes reuse `reg_write()` and return 0 if the decode misses. No delay is applied.

5. Register decode

Two decode strategies coexist:

- **Plain storage registers** (`INTR_ENABLE`, `SMBUS_CTRL`, `CTRL`, the two FIFO configs, `OVRD`, `VAL`, `TIMING0..4`, `TIMEOUT_CTRL`, `TARGET_ID`, the three timeout regs, `SMBUS_STATUS`) live in the `regmodel::RegisterMap32 regmap_` dispatch table. Their read/write masks and reset values enforce the field contract; the `switch` never needs to hand-mask them.
- **Behavioural registers** (anything with a side effect — `INTR_STATE`, `INTR_TEST`, `STATUS`, `RDATA`, `FDATA`, `FIFO_CTRL`, the computed FIFO-status/next-data regs, `ACQDATA`, `TXDATA`, `TARGET_NACK_COUNT`, `TARGET_ACK_CTRL`, `CONTROLLER_EVENTS`, `TARGET_EVENTS`) are handled explicitly in `reg_read()` / `reg_write()` and are intentionally kept **out** of the map.

`reg_read/reg_write` first switch on the behavioural offsets; on a miss they fall through to `regmap_.read()/write()`; a miss there returns `false` ($\rightarrow$ `TLM_ADDRESS_ERROR_RESPONSE`). Read-only registers appear as explicit `return true` (write-ignore) cases; write-only registers read back 0.

6. Internal data structures

Member	Type	Role
<code>fmt_</code>	<code>std::deque<fmt_entry></code>	Controller TX (FMT) FIFO
<code>rx_</code>	<code>std::deque<uint8_t></code>	Controller RX FIFO
<code>tx_</code>	<code>std::deque<uint8_t></code>	Target TX FIFO
<code>acq_</code>	<code>std::deque<acq_entry></code>	Target RX (ACQ) FIFO
<code>intr_latched_</code>	<code>uint32_t</code>	Latched W1C interrupt sources
<code>intr_force_</code>	<code>uint32_t</code>	<code>INTR_TEST</code> forced level sources
<code>controller_events_</code>	<code>uint32_t</code>	<code>CONTROLLER_EVENTS</code> (W1C)
<code>target_events_</code>	<code>uint32_t</code>	<code>TARGET_EVENTS</code> (W1C)
<code>target_ack_ctrl_</code>	<code>uint32_t</code>	<code>TARGET_ACK_CTRL.NBYTES</code>
<code>target_nack_count_</code>	<code>uint32_t</code>	Saturating 8-bit NACK counter
<code>halted_</code>	<code>bool</code>	Controller halted (awaiting event clear)
<code>bus_model_</code>	<code>i2c_bus_model_fn</code>	Controller-Mode remote-target emulator
<code>out_irq_</code> , <code>outputs_valid_</code>	<code>bool</code>	Output cache (suppress redundant <code>irq_o</code> writes)

```
fmt_entry = {byte, start, stop, readb, rcont, nakok}; acq_entry = {byte, i2c_acq_signal}.
```

7. Process and event topology

Three `SC_METHODS`, all `dont_initialize()`:

Process	Sensitivity	Responsibility
<code>reset_proc</code>	<code>rst_n_i</code>	On low level, clears all registers/FIFOs/flags, cancels the pending xfer event, schedules a recompute.
<code>recompute_method</code>	<code>recompute_event_</code>	Sole driver of <code>irq_o</code> . Writes <code>irq_active()</code> , cached to suppress redundant writes.
<code>xfer_method</code>	<code>xfer_event_</code>	Calls <code>drain_fmt()</code> — the Controller-Mode transaction engine.

Every state-changing path calls `schedule_recompute()` (`recompute_event_.notify(SC_ZERO_TIME)`). Controller work is deferred by `schedule_xfer()` (`xfer_event_.notify(xfer_delay_)`). This keeps `irq_o` glitch-free (single writer) and gives transfers a bounded, non-zero latency without a clock.

8. Behavioural algorithms

8.1 `push_fmt(fdata)` (FDATA write)

Decode the entry; if the FMT FIFO is full, set `CONTROLLER_TX_FIFO_ERROR` and drop it, else enqueue. If `ENABLEHOST` and not halted, `schedule_xfer()`. Always `schedule_recompute()`.

8.2 `drain_fmt()` (xfer engine)

Returns immediately if host disabled or halted. Walks the FMT FIFO reassembling segments:

- **START**: if a segment is already open, `execute()` it first (repeated-START), then open a new segment `{addr = byte>>1, dir = byte&1, nakok}`.
- non-START while a segment is open: for reads add to `read_len` (`byte == 0 ⇒ 256`); for writes append to the write payload; OR-in `nakok`.
- **STOP**: `execute()` the open segment; if not halted, latch `CMD_COMPLETE`.

`execute()` builds an `i2c_xfer`, calls `bus_model_` if attached, and:

- on NACK without `NAKOK`: set `CONTROLLER_EVENTS.NACK`, set `halted_`, stop draining;
- on a read ACK: push returned bytes into `rx_`, setting `RX_OVERFLOW` for each byte dropped because `rx_` is full.

A trailing open segment (no `STOP` yet) is executed at the end so reads surface promptly in this LT model.

8.3 Target back doors

`target_match(addr)` tests both `{ADDRESS0, MASK0}` and `{ADDRESS1, MASK1}` (`mask ≠ 0`). On a mismatch or with `ENABLETARGET` clear, `target_write/target_read` bump `TARGET_NACK_COUNT` and return `false`. On a match, `acq_push()` appends `Start/Data/Stop` entries (setting `TARGET_RX_FIFO_ERROR` on ACQ overflow), latches `START_DETECT/STOP_DETECT`, and (for reads) drains up to `nbytes` from `tx_`.

8.4 Status / level computation

`compute_status()`, `compute_host_fifo_status()`, `compute_target_fifo_status()`, and `level_status()` are pure functions of the FIFO sizes, thresholds, and event registers. They are evaluated on demand for reads and for interrupt aggregation; nothing is cached except the `irq_o` output value.

9. Reset implementation

`reset_proc` runs on any change of `rst_n_i` but acts only when the level is low (`return` on high). It calls `.reset(0)` on every storage register, zeroes the behavioural scalars, clears the four dequeues, cancels `xfer_event_`, and schedules a recompute so `irq_o` returns to 0. Reset is asynchronous (level-sensitive) and takes effect in the current delta cycle.

10. Loosely-timed timing model and quantum keeper

The model follows the LT discipline:

- `b_transport` **never** calls `wait()`. It only adds `access_delay_ns` to the `sc_time& delay` argument. Temporal decoupling — advancing the local time and syncing on the global quantum — is the **initiator's** responsibility.
- The reference test-bench driver owns a `tlm_utils::tlm_quantumkeeper`: it seeds the payload delay from `qk.get_local_time()`, calls `b_transport`, then `qk.set_and_sync(t)` to let the kernel advance when the quantum is exceeded.
- The modelled transaction latency `xfer_delay_ns` is not spent inside `b_transport`; instead `schedule_xfer()` posts `xfer_event_` that far in the future, and `xfer_method` runs `drain_fmt()` when the kernel reaches that time. This gives Controller-Mode transfers a realistic, observable latency while keeping the register path decoupled.

This split (annotate-in- `b_transport`, execute-on-event) is the same pattern used by the `i3c_controller` and `uart` models.

11. Interrupt aggregation

```
intr_state_read() = level_status() | intr_latched_ | intr_force_. irq_active() =
(intr_state_read() & intr_enable_.read()) != 0.
```

- `level_status()` recomputes the level bits (FMT/RX/ACQ/TX thresholds, `CONTROLLER_HALT` from `controller_events_`, `TX_STRETCH` from `target_events_`).
- Writing `INTR_STATE` clears latched bits via `regmodel::apply_w1c(intr_latched_, data, W1C_MASK)`.
- Writing `INTR_TEST` latches the event bits (`data & W1C_MASK`) and holds the level bits (`intr_force_ = data & LEVEL_MASK`).
- `recompute_method` is the only writer of `irq_o`.

12. Debug and verification hooks

Hook	Purpose
<code>set_bus_model(fn)</code>	Attach the Controller-Mode remote-target emulator.
<code>target_write(addr, data, stop=true)</code>	Emulate an external controller writing this device.
<code>target_read(addr, nbytes, out, stop=true)</code>	Emulate an external controller reading this device.
<code>dbg_reg(off)</code>	Side-effect-free register peek (no FIFO pop, no read-clear).
<code>dbg_fmt_count() / dbg_rx_count() / dbg_tx_count() / dbg_acq_count()</code>	FIFO occupancy.
<code>dump_state(os)</code>	Human-readable snapshot (CCI params, CTRL, INTR state, FIFO levels, irq).

`dbg_reg` mirrors `reg_read` for the computed/behavioural registers but never mutates state, which is what makes `transport_dbg` reads safe for a debugger or checker.

13. Integration with `smc_top`

```
smc::i2c_controller_cfg cfg;           // optionally override defaults
smc::i2c_controller i2c0("i2c0", cfg);

fabric_initiator.bind(i2c0.reg_socket); // AXI4-Lite-style target socket
i2c0.rst_n_i(rst_n);
i2c0.irq_o(i2c0_irq);                 // route to the PLIC input

// Optional: preset FIFO depths before construction
broker.set_preset_cci_value("top.i2c0.rx_fifo_depth", cci::cci_value(128u));
```

The base address and instance spacing (`0x200`) come from the `i2c_wrap` address map; the platform decoder routes `[base, base+0x200)` to this socket. For multiple cores, instantiate `i2c0..i2cN` and give each a distinct hierarchical name so CCI param paths stay unique.

14. Modeling decisions and trade-offs

Decision	Rationale
Model one <code>i2c</code> core, not the <code>i2c_wrap</code> bundle	Matches the natural reuse unit; the wrapper enable register is trivial and platform-specific (like <code>uart</code> vs <code>uart_wrap</code>).
Segment reassembly in <code>drain_fmt</code> rather than byte-FSM	LT models care about transaction outcomes, not SCL edges; a bus-model callback per segment is far simpler to drive and check.
Bus behaviour supplied by a test-bench callback	Keeps the model free of any particular remote-target policy; benches can emulate ACK/NACK/data arbitrarily.
<code>TIMINGx</code> / timeouts / <code>OVRD</code> / <code>VAL</code> as storage	Analog/serial timing is not meaningful at LT; firmware can still read back what it wrote.
Single-driver <code>irq_o</code> via <code>recompute_method</code>	Prevents multiple-driver races and delta glitches; consistent with <code>uart / plic / reset_unit</code> .
Immutable FIFO-depth params	Depth is a build-time RTL parameter; making it immutable models that faithfully and lets the broker preset it before construction.

15. Known limitations and future work

- **No bit-level bus:** SCL/SDA waveform, clock stretching, arbitration, and glitch filtering are not modelled. `SCL_INTERFERENCE`, `SDA_INTERFERENCE`, `STRETCH_TIMEOUT`, `SDA_UNSTABLE`, `UNEXP_STOP`, `HOST_TIMEOUT`, and `ACQ_STRETCH` are therefore reachable only through `INTR_TEST`.
- **SMBus** alert/suspend and the multi-controller monitor path are storage-only; add functional behaviour if firmware bring-up needs it.

- **RCONT** is accepted but not separately modelled (reads always return the requested length from the bus model).
- **Timeouts** do not fire; if timeout-driven firmware paths need coverage, a future revision can arm an `sc_event` from `TIMEOUT_CTRL` / the host/target timeout regs.